

MATERIAŁ DLA DYDAKTYKÓW I PRAKTYKÓW AI

Pętle agentowe: od promptu do autonomicznego procesu

Przegląd stanu wiedzy, wzorce i studium przypadku, które sam odpalisz

TEMAT	AUTOR
Pętle agentowe (agentic AI)	Błażej Strus
KATEGORIA	DATA
Loop engineering	lipiec 2026

► W SKRÓCIE

Pewnie znasz to uczucie. Prosisz AI o coś, dostajesz prawie-dobrą odpowiedź, poprawiasz, pytasz znowu, i tak w kółko, wszystko ręcznie. **Pętla agentowa** zdejmuję tę pracę z Ciebie. Model sam planuje, sięga po narzędzia, sprawdza własny wynik i poprawia go, aż trafi. To nie marketing, tylko dobrze udokumentowany wzorzec inżynieryjny. Pisałem ten tekst z myślą o dwóch osobach naraz, o koledze, który uczy tego studentów, i o praktyku, który po prostu chce, żeby jego agent działał. Tłumaczę rzecz po ludzku, pokazuję żywy przykład w Pythonie, dokładam własny benchmark, a na końcu odsłaniam nowość, czyli pętlę, która potrafi ulepszać samą siebie.

CZEGO SIĘ NAUCZYSZ

- 01 Odróżnisz zadanie, workflow i agenta oraz rozpoznasz, kiedy pętla się opłaca, a kiedy jest pułapką.
- 02 Poznasz wzorce projektowe (Ng, Anthropic, OpenAI) i ich naukowy rodowód: ReAct, Reflexion, Self-Refine.
- 03 Zrozumiesz, dlaczego weryfikacja musi być zewnętrzna, i prześledzisz działający przykład: nietrywialny problem matematyczny z funkcją kontrolną w Pythonie.

1 Czym jest pętla agentowa

INFORMACJA

i

Jak to czytać. Nie musisz śledzić wszystkich prac, które przytaczam, żeby z tego skorzystać. Sekcje 1–3 dają intuicję i słownik pojęć, sekcja 4 to mapa źródeł dla ciekawych, a 5–8 to konkret, który najlepiej pokazać studentom albo odpalić samemu z repozytorium. Jeśli się spieszysz, zacznij od sekcji 5 i 7, reszta poczeka.

Większość z nas używa AI w trybie **jeden prompt naraz**: wpisujemy polecenie, czekamy, poprawiamy, pytamy znowu. Działa, ale to my jesteśmy silnikiem – gdy przestajemy klikać, wszystko się zatrzymuje. Pętla działa inaczej. Dajesz cel raz, a model sam go realizuje: **planuje, wykonuje pracę narzędziami, sprawdza własny wynik i poprawia**, aż cel zostanie spełniony albo zadziała warunek stopu. Ty wychodzisz z pętli, a praca toczy się dalej.

Dziś mamy jedną, prostą definicję pętli agentowej. Anthropic, po wcześniejszym rozróżnieniu workflow i agentów, opisuje agenta po prostu jako **LLM autonomicznie używający narzędzi w pętli**. Minimalny, ale precyzyjny rdzeń – i dobra intuicja na start.

► DEFINICJA • PĘTLA AGENTOWA

Cykl, w którym model językowy sam planuje, wykonuje działania za pomocą narzędzi, weryfikuje wynik względem celu i iteruje, aż cel zostanie spełniony lub zadziała warunek stopu. W odróżnieniu od pojedynczego promptu człowiek nie steruje każdym krokiem.

INFORMACJA

i

Kanoniczny rdzeń pętli to pięć faz:

- DISCOVER (co trzeba zrobić),
- PLAN (jak),
- EXECUTE (wykonanie),
- VERIFY (sprawdzenie względem celu),
- ITERATE (powrót z wynikiem).

Trzy z nich – weryfikacja, stan i warunek stopu – decydują, czy pętla działa, czy tylko miełe tokeny.

2 Taksonomia: zadanie, workflow, agent

Zanim zbudujesz pętlę, nazwij problem. Barry Zhang z Anthropic proponuje trzy szczeble. **Zadanie** to pojedyncze wywołanie modelu. **Workflow** to wiele wywołań w przepływie sterowania, który projektuje człowiek. W **agencie** to model jest właścicielem przepływu – sam decyduje o kolejnych krokach. Pierwsza zasada brzmi: **nie buduj agentów do wszystkiego**.

Poziom	Kto steruje przepływem	Kiedy stosować
Zadanie	Człowiek (jeden prompt)	Klasyfikacja, ekstrakcja, streszczenie
Workflow	Człowiek (kod, predefiniowane ścieżki)	Da się zmapować drzewo decyzyjne

Agent	Model (decyzje w czasie działania)	Zadanie niejednoznaczne, o wysokiej wartości
-------	------------------------------------	--

Reguła kciuka Zhanga: jeśli da się zmapować drzewo decyzyjne, zbuduj workflow – dostaniesz więcej trafności, kontroli i niższy koszt niż da jakikolwiek agent. Agent kosztuje, bo eksploruje.

3 Wzorce projektowe: słownik pojęć

Dwie taksonomie organizują dziś myślenie o pętlach. Poniżej tłumaczę każdy wzorzec po kolei, bo wracają w dokumentacji i artykułach jako skróty myślowe, które warto rozumieć dosłownie.

Pięć wzorców workflow (Anthropic)

Workflow to z góry zaprojektowany przepływ wielu wywołań modelu, którym steruje człowiek (kod). Anthropic wyróżnia pięć podstawowych układów.

Prompt chaining (łańcuch promptów) – rozbicie zadania na sekwencję kroków, w której wyjście jednego wywołania jest wejściem następnego. Przykład: najpierw wygeneruj konspekt, potem rozwiń go w tekst. Stosuj, gdy zadanie ma jasne, następujące po sobie etapy.

Routing – pierwszy model klasyfikuje wejście i kieruje je do wyspecjalizowanej ścieżki. Przykład: zgłoszenie trafia do obsługi zwrotów, pomocy technicznej albo sprzedaży. Stosuj, gdy istnieją rozłączne kategorie wymagające innej obsługi.

Parallelization (zrównoleglenie) – kilka wywołań pracuje równolegle: albo dzielimy zadanie na niezależne części, albo wykonujemy tę samą pracę kilka razy i głosujemy nad wynikiem. Przykład: trzy niezależne oceny tekstu i agregacja. Stosuj dla szybkości lub większej pewności.

Orchestrator-workers – centralny model dynamicznie dzieli zadanie, deleguje podzadania do workerów i scala wyniki. W odróżnieniu od zrównoleglenia podział nie jest ustalony z góry, lecz powstaje w trakcie. Przykład: złożona zmiana rozłożona na wiele plików kodu.

Evaluator-optimizer – jeden model generuje wynik, drugi go ocenia i zwraca feedback, i tak w pętli, aż jakość wystarczy. To „pętla” w czystej postaci. Stosuj, gdy masz jasne kryteria oceny i gdy iteracyjne dopracowanie realnie poprawia wynik.

Cztery wzorce agentowe (Ng)

Andrew Ng nazwał cztery wzorce, które najczęściej podnoszą jakość systemów agentowych. Można je łączyć w jednym systemie.

Reflection (refleksja) – model krytykuje własny wynik i poprawia go w kolejnym przebiegu, jak automatyczny recenzent. Uwaga: skuteczna głównie wtedy, gdy krytyka opiera się na sygnale zewnętrznym (patrz sekcja 5).

Tool use (użycie narzędzi) – model sięga poza własną wiedzę: wywołuje API, uruchamia kod, czyta bazy. To zamienia generator tekstu w system, który realnie działa.

Planning (planowanie) – model rozkłada cel na kroki i adaptuje plan, gdy któryś krok zawiedzie. Pozwala obsłużyć zadania wieloetapowe zamiast rzucać się od razu na rozwiązanie.

Multi-agent collaboration – kilku wyspecjalizowanych agentów dzieli pracę (np. autor i recenzent). To tu rodzi się wzorzec maker/checker, kluczowy dla weryfikacji.

4 Naukowy rodowód: metody, które trzeba znać

Popularne teksty mówią o „pętli” bez nazwisk. Zebrałem więc recenzowane podstawy w jednym miejscu, żebyś nie musiał ich szukać, i opisałem, co każda metoda rozwiązała oraz jak działa. Nie zakładam, że przeczytałeś każdą z tych prac, spokojnie. Punktem wyjścia jest to, jak w ogóle skłaniamy model do rozumowania krok po kroku.

Chain-of-Thought (Wei i in., 2022) – zachęcenie modelu, by rozpisał kolejne kroki rozumowania, zamiast od razu podać wynik, wyraźnie poprawia trafność w zadaniach wieloetapowych. To fundament, na którym budują kolejne metody.

ReAct (Yao i in., 2022) – przeplata rozumowanie z działaniem w cyklu Myśl → Akcja → Obserwacja. Model na przemian rozpisuje krok rozumowania i wykonuje akcję narzędziem (np. wyszukanie), a obserwację z otoczenia wciąga z powrotem do rozumowania. Kontakt z „prawdą z otoczenia” ogranicza halucynacje typowe dla samego łańcucha myśli. To rdzeń pary PLAN/EXECUTE.

Reflexion (Shinn i in., 2023) – po nieudanej próbie model zapisuje słowną autorefleksję („dlaczego się nie udało”) do pamięci epizodycznej i wykorzystuje ją w następnym podejściu – bez douczania wag, samym językiem. To naukowy rodowód faz VERIFY i STATE: uczenie się z porażek między próbami.

Self-Refine (Madaan i in., 2023) – jeden model w trzech rolach: generuje wynik, sam daje sobie feedback, poprawia – i powtarza, aż kryteria są spełnione. Pokazuje, że iteracyjne dopracowanie pomaga, ale (jak zobaczymy) tylko wtedy, gdy feedback jest trafny.

CRITIC (Gou i in., 2024) – krok dalej: samokorekta nie z introspekcji, lecz przez narzędzia zewnętrzne (interpreter, wyszukiwarka, weryfikator). Model konfrontuje wynik z obiektywnym narzędziem i poprawia. To bezpośredni rodowód „bramy zewnętrznej” z naszego studium przypadku.

Chain-of-Verification (Dhuliawala i in., 2023) – model najpierw pisze szkic, potem układa pytania kontrolne, odpowiada na nie niezależnie (by nie sugerować się szkicem) i na końcu redaguje zweryfikowaną wersję. Obniża halucynacje, bo rozdziela tworzenie od sprawdzania nawet w obrębie jednego modelu.

Tree of Thoughts (Yao i in., 2023) – zamiast jednej ścieżki rozumowania model rozważa wiele gałęzi, ocenia je i przeszukuje jak drzewo decyzyjne. To rozwinięcie wzorca planowania i przeszukiwania.

Te metody mapują się wprost na fazy pętli: Chain-of-Thought i ReAct na PLAN/EXECUTE, Reflexion na STATE, a Self-Refine, CRITIC i CoVe na VERIFY. Wspólny wniosek prowadzi do sekcji 5: najmocniejsza weryfikacja jest zewnętrzna.

5 Sedno: weryfikacja

Weryfikacja jest sercem pętli. Bez realnego sprawdzenia wyniku nie ma pętli – jest tylko model, który w kółko przyznaje sam sobie rację. Część metod z sekcji 4 (Self-Refine, refleksja) zakłada samokorektę wewnętrzną (intrinsic self-correction): model poprawia własną odpowiedź wyłącznie na podstawie swojego osądu, bez sygnału z zewnątrz. I tu pojawia się najważniejsze ustalenie naukowe, którego teksty popularne nie podają.

×	BŁĄD Antywzorzec: model ocenia własną pracę. Huang i in. (ICLR 2024) wykazali, że bez zewnętrznego feedbacku LLM-y nie poprawiają własnego rozumowania, a wynik po samokorekcie potrafi się pogorszyć. Stechly, Valmeekam i Kambhampati (2024) potwierdzają ograniczenia samo-weryfikacji w rozumowaniu i planowaniu. Model, który wykonał pracę, jest zbyt pobłażliwym sędzią.
✓	DOBRA PRAKTYKA Brama weryfikująca musi być zewnętrzna i obiektywna: test, typecheck, build, walidator lub twardy warunek liczbowy. Rozdziel wykonawcę od kontrolera (wzorzec maker/checker) i każ agentowi pokazać dowód – wynik testu, polecenie i jego output – zamiast deklarować sukces.

To uzasadnia oba elementy naraz: tezę „verify is the heart of the loop” oraz wzorzec rozdzielania wykonawcy i recenzenta. Kodowanie jest dobrym przypadkiem agentowym właśnie dlatego, że testy jednostkowe dostarczają taniej, zewnętrznej weryfikacji.

6 Koszty i ciche awarie

Pętle działają na tokenach, a koszt nie rośnie liniowo – przy każdej iteracji rośnie też kontekst wysyłany ponownie do modelu. Anthropic podaje twarde liczby z własnych wdrożeń.

Tryb pracy	Zużycie tokenów	Wniosek
Pojedynczy chat	×1 (odniesienie)	Tani, przewidywalny
Agent w pętli	≈ ×4	Uzasadniony przy realnej autonomii
System wieloagentowy	≈ ×15	Tylko zadania o wysokiej wartości

!	UWAGA Metryka, którą warto śledzić, to koszt na zaakceptowaną zmianę, nie liczba tokenów. Pętla bez twardej bramy nie pada z hukiem – cicho liczy rachunek (tzw. pętla Ralpha Wigguma: agent uznaje zadanie za skończone zbyt wcześnie). Każda poważna pętla potrzebuje warunku stopu i limitu iteracji.
---	---

7 Studium przypadku: pętla, którą sam odpalisz

Czas zejść z teorii na ziemię, bo najłatwiej tłumaczy się na czymś, co odpala się jednym poleceniem. Wybierzmy zadanie, którego model nie zna z pamięci, bo losujemy je za każdym razem: **subset-sum**. Masz zbiór liczb i cel. Znajdź podzbiór, który sumuje się dokładnie do celu. Brzmi niewinnie, ale to jeden z klasycznych trudnych problemów, NP-zupełny: **znaleźć rozwiązanie bywa koszmarnie trudno, ale sprawdzić podanego kandydata trywialnie łatwo**. Ta przepaść między „znaleźć” a „sprawdzić” to asymetria generator–weryfikator, i właśnie ona tłumaczy, dlaczego brama musi stać poza modelem.

✓	DOBRA PRAKTYKA
---	------------------------------

Cały kod jest otwarty. Pętla, weryfikatory, pięć problemów i benchmark z sekcji 8 leżą w repozytorium github.com/blablast/agent-loop. Klonujesz, robisz „uv sync” i odpalasz „uv run python examples/run.py --problem subset_sum”. Jest też wariant offline, bez klucza API, więc możesz dać studentom coś, co realnie działa na zajęciach, a nie tylko slajd.

Krok 1: weryfikator, czyli brama

Weryfikator jest krótki, deterministyczny i nie ma pojęcia, co model „myśli”. Sprawdza dwie rzeczy: czy kandydat składa się z liczb ze zbioru i czy sumuje się do celu. Gdy nie – zwraca konkretną wskazówkę (np. „różnica -6”), z której skorzysta następna próba.

```
def verify(numbers, target, candidate):
    if any(x not in numbers for x in candidate):
        return False, "element spoza zbioru"
    if (total := sum(candidate)) != target:
        return False, f"różnica {target - total:+d}"
    return True, "OK"
```

listing 1 – brama: zwraca prawdę/fałsz oraz konkretną wskazówkę.

I tu pointa. `verify([8, 11, 18, 27], 64)` zwraca `(True, „OK”)`, ale prawdopodobny błędny strzał `verify([6, 8, 11, 18, 27], 64)` zwraca `(False, „różnica -6”)` – natychmiast, bez dyskusji. Model może być w stu procentach przekonany, że ma rację. Brama i tak nie przepuści złej odpowiedzi. Tego sama pewność modelu nie daje (Huang i in., 2024).

Krok 2: pętla, czyli maker proponuje, brama rozstrzyga

Pętla tylko spina to w całość: prosi makera o kandydata, daje go bramie, a gdy brama odmawia – zapisuje wskazówkę i prosi o kolejną próbę. Widać tu wszystkie cztery elementy z sekcji 1 naraz: stan, weryfikację, iterację i podwójny warunek stopu.

```
def loop(numbers, target, maker, max_iters=40):
    history = [] # STATE: attempts
    for _ in range(max_iters): # STOP: hard limit
        candidate = maker(numbers, target, history)
        ok, feedback = verify(numbers, target, candidate)
        if ok:
            return candidate # STOP: success
        history.append((candidate, feedback))
    return None
```

listing 2 – szkielet pętli: cała jej moc bierze się z zaufanej bramy.

Krok 3: maker na modelu Gemini (i dlaczego bez MCP)

Makerem może być cokolwiek, co proponuje kandydatów – nawet zwykłe przeszukiwanie. W praktyce podstawiamy model. Tu Gemini: wskazówki z bramy wkładamy do promptu kolejnej próby, więc model uczy się z własnych pomyłek w locie. I ważna rzecz: **nie potrzebujemy do tego serwera MCP**. Weryfikator to nie narzędzie, które wywołuje model – to zewnętrzny test w naszym kodzie. O to właśnie chodzi: brama ma być niezależna od modelu.

```

from google import genai
from google.genai import types

def gemini_maker(numbers, target, history):
    client = genai.Client() # reads GEMINI_API_KEY
    tries = "\n".join(f"{c} -> {fb}" for c, fb in history)
    prompt = f"S={numbers}, target={target}. Tried:\n{tries}"
    resp = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt,
        config=types.GenerateContentConfig(
            thinking_config=types.ThinkingConfig(
                thinking_level="high"),
        )
    )
    return parse_list(resp.text) # e.g. [3, 7, 12]

```

listing 3 – maker na Gemini (gemini-3.5-flash), klucz z GEMINI_API_KEY.

Dobry start to gemini-3.5-flash z thinking_level „high” – szybki i tańszy, a pętla robi wiele wywołań. Gdy trafi się twardszy przypadek, sięgasz po mocniejszy gemini-3.1-pro. Reszta kodu się nie zmienia – wymieniasz tylko makera.

Najważniejsze: przeszukiwanie, heurystyka i Gemini są wymienne, bo o sukcesie decyduje brama, nie autor rozwiązania. To dokładnie wzorzec maker/checker. I dlatego programowanie jest sztandarowym zadaniem dla agentów: testy dają darmową, obiektywną bramę.

Cztery zasady na koniec

1. Zaczynaj od najprostszego rozwiązania. Agent to ostateczność, nie domysłność.
2. Pętla ma sens tylko wtedy, gdy coś potrafi automatycznie odrzucić zły wynik.
3. Weryfikacja zawsze zewnętrzna. Pewność modelu to nie to samo co poprawność.
4. Patrz na koszt zaakceptowanej zmiany i zawsze ustaw twardy warunek stopu.

8 Wyniki eksperymentów: sweep benchmarkowy

Studium z sekcji 7 pokazało pętlę na jednym przykładzie. Ale skoro cała teza brzmi „brama, nie autor, decyduje o sukcesie”, to wypada ją zmierzyć. Zamiast jednego przebiegu odpaliłem **sweep**: ten sam szkielet pętli i ten sam zewnętrzny weryfikator, ale przez siatkę modeli, problemów i losowań. Łącznie **800 przebiegów** – 9 modeli × 5 problemów × 2 poziomy „myślenia” (high / minimal) × 10 losowań (seed), z twardym limitem 12 iteracji. Wszystkie modele są lokalne (Ollama), więc porównujemy zdolność pętli, a nie ceny API. Każdy problem to inny kształt kandydata i inna, deterministyczna brama. Subset-sum ze studium jest tylko jednym z pięciu.

INFORMACJA

i

Plan eksperymentu. Pięć problemów (subset_sum, optimization, regex_golf, reverse_engineering, maze), dziewięć lokalnych modeli (od lfm2.5 i qwen3:4b po gemma4:31b i deepseek-r1:32b), po 10 seedów na komórkę, max 12 iteracji. Mierzmy trzy metryki niezależne od sprzętu: odsetek rozwiązań (succ%), medianę liczby iteracji potrzebnych do sukcesu i medianę zużytych tokenów. Czas generowania świadomie pomijamy – zależy od chwilowego obciążenia GPU, nie od jakości pętli.

Co modele potrafią rozwiązać: skuteczność wg problemu

Najpierw poziom trudności. Uśredniając po wszystkich modelach i ustawieniach, pięć problemów układa się w wyraźną drabinę – od niemal zawsze rozwiązywalnego subset-sum po prawie nierozwiązywalne optimization i reverse_engineering.

Problem	Skuteczność (wszystkie modele)	Charakter
subset_sum	93.1%	kandydat trywialny do sprawdzenia – problem-podłoga
regex_golf	73.8%	krótka przestrzeń, brama liczbowa – łatwy
maze	37.5%	planowanie ścieżki – średni, silnie zależny od modelu
reverse_engineering	10.6%	ukryta funkcja – twardy, wąski feedback
optimization	8.8%	ciągła minimalizacja straty – najtwardszy

To pierwszy wniosek, który łatwo przeoczyć: **subset-sum ze studium przypadku jest najłatwiejszym z całego zestawu** – przy myśleniu „high” rozwiązuje go w 100% każdy z dziewięciu modeli. Dobry do nauki, ale prawdziwym testem pętli są optimization i reverse_engineering, gdzie prawie wszystko upada. I tu pojawia się rzecz najważniejsza dla całego materiału.

DOBRA PRAKTYKA



Brama trzyma nawet tam, gdzie model przegrywa. Na optimization i reverse_engineering modele wygenerowały łącznie kilkanaście tysięcy poprawnie sparsowanych kandydatów i dostawały konkretny feedback („loss 19.1”, „błędne wejście -7”) – a mimo to weryfikator nie przepuścił ani jednej złej odpowiedzi. Pętla nie „poddaje się” i nie oszukuje: gwarantuje **poprawność, nie zdolność**. Słaby model nie staje się mądrzejszy – staje się jedynie niezdolny do oddania błędnego wyniku jako dobrego. To dokładnie asymetria generator-weryfikator z sekcji 7, potwierdzona na 1600 przebiegach.

Czy iteracja naprawdę pomaga: feedback w liczbach

Gdyby pętla była tylko „ponawianiem”, sukcesy padałyby zawsze za pierwszym strzałem. Tymczasem z 358 udanych przebiegów **62% udaje się od razu, ale 38% dopiero po co najmniej dwóch iteracjach** – z ogonem sięgającym pełnych 12 prób. Ponad jedna trzecia zwycięstw to bezpośrednia zasługa wskazówek z bramy wracających do promptu: Reflexion z sekcji 4 w działaniu, a nie na slajdzie.

Mocniejszy model czy więcej myślenia?

Włączenie trybu „high” podnosi średnią skuteczność z 37% do 52% (na modelach, które mają oba tryby). Myślenie pomaga – ale nie monotonicznie. gpt-oss na reverse_engineering **traci na myśleniu** (80% → 40%): klasyczne „przemyslenie”. Skalowanie rozmiaru też daje zaskakująco mało – deepseek 14b→32b to tylko 42%→46%, a qwen3 4b→14b 40%→46%. Na tych zadaniach o wyniku decyduje raczej brama i iteracja niż same parametry.

!

UWAGA

Więcej mocy nie jest darmowe ani pewne. Dwa razy większy model bywa marginalnie lepszy, a więcej „myślenia” potrafi zaszkodzić (gpt-oss). Zarazem koszt tokenowy różni się między modelami około dwudziestokrotnie: mediana na jeden udany przebieg to od ~800 tokenów (lfn2.5) do ~9700 (gemma4:26b). Dlatego metryką jest **koszt na zaakceptowany wynik**, nie sam odsetek sukcesów – dokładnie jak w sekcji 6.

Tabela: ranking modeli – skuteczność na wszystkich problemach i mediana tokenów na sukces.

Model	Skuteczność (wszystkie problemy)	Mediana tok. / sukces
gpt-oss:latest	74%	3914
gemma4:31b	61%	1616
gemma4:26b	60%	3052
deepseek-r1:32b	46%	2328
deepseek-r1:14b	42%	2275
qwen3:4b	39%	4474
qwen3:14b	31%	2217
gemma4:latest	29%	1387
lfn2.5:latest	20%	787

Pełne wyniki wg problemu

Poniżej komplet 80 komórek (problem × model × tryb), po 10 seedów każda. Kolumna **succ%** to odsetek rozwiązań, **mediana iter.** to mediana liczby iteracji, gdzie przebieg nierozwiązany licz się jako 12, a **mediana tok.** to mediana sumy tokenów na przebieg. deepseek uruchamiałem tylko w trybie „high”.

Problem: subset_sum

Model	Myślenie	succ%	mediana iter.	mediana tok.
gpt-oss:latest	high	100	1	521.0
gpt-oss:latest	minimal	100	1	548.0
gemma4:31b	high	100	1	1 050.0
gemma4:31b	minimal	100	1	294.5
gemma4:26b	high	100	1	1 846.0
gemma4:26b	minimal	100	1	340.5
deepseek-r1:32b	high	100	1	1 640.5
deepseek-r1:14b	high	100	1	1 427.0
qwen3:4b	high	100	1	3 051.5
qwen3:4b	minimal	100	1	2 549.5
qwen3:14b	high	100	1	1 889.0

qwen3:14b	minimal	40	12	5 909.5
gemma4:latest	high	100	1	1 124.5
gemma4:latest	minimal	50	11	4 064.0
lfm2.5:latest	high	100	1	787.0
lfm2.5:latest	minimal	100	1	778.5

Problem: optimization

Model	Myślenie	succ%	mediana iter.	mediana tok.
gpt-oss:latest	high	0	12	12 447.5
gpt-oss:latest	minimal	20	12	13 167.5
gemma4:31b	high	50	11.5	15 831.5
gemma4:31b	minimal	0	12	3 715.0
gemma4:26b	high	40	12	37 089.0
gemma4:26b	minimal	20	12	3 686.0
deepseek-r1:32b	high	0	12	23 985.5
deepseek-r1:14b	high	0	12	17 215.0
qwen3:4b	high	0	12	44 660.0
qwen3:4b	minimal	0	12	82 228.5
qwen3:14b	high	0	12	21 138.0
qwen3:14b	minimal	0	12	3 884.0
gemma4:latest	high	10	12	11 941.0
gemma4:latest	minimal	0	12	3 684.5
lfm2.5:latest	high	0	12	12 557.5
lfm2.5:latest	minimal	0	12	47 767.5

Problem: regex_golf

Model	Myślenie	succ%	mediana iter.	mediana tok.
gpt-oss:latest	high	100	1	1 854.5
gpt-oss:latest	minimal	100	1	2 284.5
gemma4:31b	high	100	1	1 686.5
gemma4:31b	minimal	90	1	261.0
gemma4:26b	high	100	1	5 411.5
gemma4:26b	minimal	90	2	989.0
deepseek-r1:32b	high	90	1	2 981.0
deepseek-r1:14b	high	100	1	3 042.5
qwen3:4b	high	90	1	8 082.0

qwen3:4b	minimal	90	1	6 793.0
qwen3:14b	high	80	1.5	6 459.0
qwen3:14b	minimal	40	12	6 105.0
gemma4:latest	high	90	1	2 192.0
gemma4:latest	minimal	20	12	6 829.0
lfm2.5:latest	high	0	12	23 200.0
lfm2.5:latest	minimal	0	12	37 440.0

Problem: reverse_engineering

Model	Myślenie	succ%	mediana iter.	mediana tok.
gpt-oss:latest	high	40	10.5	116 368.0
gpt-oss:latest	minimal	80	5.5	82 334.5
gemma4:31b	high	10	12	104 214.0
gemma4:31b	minimal	0	12	13 578.5
gemma4:26b	high	40	12	250 312.0
gemma4:26b	minimal	0	12	63 436.0
deepseek-r1:32b	high	0	12	125 662.0
deepseek-r1:14b	high	0	12	137 346.0
qwen3:4b	high	0	12	188 898.0
qwen3:4b	minimal	0	12	192 484.0
qwen3:14b	high	0	12	141 372.0
qwen3:14b	minimal	0	12	13 065.0
gemma4:latest	high	0	12	64 591.5
gemma4:latest	minimal	0	12	12 444.5
lfm2.5:latest	high	0	12	800 701.0
lfm2.5:latest	minimal	0	12	762 882.0

Problem: maze

Model	Myślenie	succ%	mediana iter.	mediana tok.
gpt-oss:latest	high	100	7	20 070.5
gpt-oss:latest	minimal	100	7	19 203.0
gemma4:31b	high	100	7	14 154.0
gemma4:31b	minimal	60	11	5 252.0
gemma4:26b	high	100	7	37 048.0
gemma4:26b	minimal	10	12	6 233.5
deepseek-r1:32b	high	40	12	55 624.5

deepseek-r1:14b	high	10	12	48 622.0
qwen3:4b	high	10	12	118 324.0
qwen3:4b	minimal	0	12	140 962.0
qwen3:14b	high	50	10.5	51 003.5
qwen3:14b	minimal	0	12	5 601.0
gemma4:latest	high	20	12	35 730.5
gemma4:latest	minimal	0	12	5 545.0
lfm2.5:latest	high	0	12	47 469.5
lfm2.5:latest	minimal	0	12	70 080.0

Wniosek spina cały materiał: wyniki potwierdzają tezę z sekcji 5 twardymi liczbami. Pętla nie zrobiła ze słabego modelu mocnego – zrobiła z niego model, który **nie oddaje błędnej odpowiedzi jako poprawnej**, a tam gdzie potrafił, iteracyjny feedback realnie skracał drogę do rozwiązania. Zewnętrzna, obiektywna brama i twardy warunek stopu okazują się ważniejsze niż rozmiar modelu czy ilość „myślenia” – i to jest lekcja, którą warto zostawić sobie i innym przed sekcją o pętlach ulepszających same instrukcje.

9 Nowość: pętla, która ulepsza własny skill

Do tej pory pętla rozwiązywała zadanie. A co, gdyby skierować ją na same instrukcje, którymi karmimy agenta? To najświeższy nurt w tej dziedzinie (2025–2026) i warto o tym napisać, bo właśnie tam przesuwa się rola inżyniera.

Każdy, kto kiedyś pisał agentowi instrukcję (tzw. **skill** – plik z procedurą postępowania), zna ten ból: poprawiasz jedno, psujesz drugie, i nie wiesz, która linijka za co odpowiada. Ręczne dłubanie w tym tekście nie skaluje się. Nowe frameworki proponują coś, co brzmi odważnie, a działa: potraktujmy plik skilla jak **trenowalny stan** i ulepszajmy go pętlą – bez ruszania wag modelu.

SkillOpt: trening pliku instrukcji

SkillOpt (Microsoft, open-source) robi to z dyscypliną treningu sieci. W kółko: uruchamia agenta na zestawie zadań i zbiera przebiegi. Drugi model-optymalizator czyta, co zawiodło, proponuje drobne edycje skilla (dodaj/usuń/zamień), ograniczone „tekstowym learning rate”, żeby nie rozwalić całości. A potem brama: zmianę przyjmuje tylko, jeśli realnie poprawia wynik na osobnym zbiorze testowym. Odrzucone edycje trafiają do bufora, żeby błędy się nie powtarzały. Efekt: skill robi się tylko lepszy, nigdy gorszy, a na wyjściu masz mały plik, który nic nie kosztuje przy uruchomieniu. W 52 testowanych konfiguracjach był najlepszy lub równy najlepszemu, a na GPT-5.5 podniósł skuteczność o ponad 23 punkty.

INFORMACJA

i

Pętla SkillOpt: rollout (uruchom zadania) → refleksja (co zawiodło) → ograniczone edycje tekstu → brama held-out (przyjmij tylko, jeśli poprawia) → bufor odrzuconych. To znajome fazy weryfikacji i iteracji, tylko zastosowane do samej instrukcji, nie do zadania.

GEPA: ewolucja przez refleksję

GEPA (Genetic-Pareto) podchodzi do tego od strony ewolucyjnej. Trzyma kilka wersji promptu naraz, czyta ślady wykonania, po ludzku diagnozuje, co poszło nie tak, i wybiera najlepsze warianty z tzw. frontu Pareto. Co ważne, robi to oszczędnie – w pracy z ICLR 2026 bije metody uczenia ze wzmocnieniem przy około 35 razy mniejszej liczbie przebiegów – i działa z popularnym frameworkiem DSPy. EvoSkill i TextGrad to pokrewne pomysły, m.in. dla skilli agentów kodujących.

! UWAGA

To nie magia. Te systemy działają tylko tam, gdzie istnieje weryfikowalny sygnał i czysty zbiór held-out – te same warunki, których wymaga każda dobra pętla. Na zadaniach subiektywnych i otwartych nie zadziałają. Koszt płacisz z góry (optymalizator czyta mnóstwo śladów), ale gotowy skill nie obciąża już inferencji.

Zwróć uwagę na puentę: to ta sama myśl, która przewija się przez cały materiał, tylko piętro wyżej. **Nawet samodoskonalenie jest bezpieczne wyłącznie za obiektywną, zewnętrzną bramą.** Dla dydaktyka to ważny sygnał: rola inżyniera przesuwana się od majstrowania przy pojedynczych zdaniach w prompcie do projektowania systemów – z metrykami, pamięcią i warunkami stopu. I to właśnie warto dziś pokazywać studentom.

► CO ZAPAMIĘTAĆ

Pętla agentowa to LLM używający narzędzi w cyklu plan – działanie – weryfikacja – iteracja. Jej rdzeń ma recenzowany rodowód (ReAct, Reflexion, Self-Refine), a jeden warunek decyduje o wszystkim. Weryfikacja musi być zewnętrzna, bo pewność modelu to nie poprawność. Studium subset-sum pokazuje to dotykalnie, bo znaleźć rozwiązanie jest trudno, a sprawdzić kandydata łatwo. Pętli nie buduje się do wszystkiego. Liczą się powtarzalność, obiektywna brama i twardy warunek stopu, a koszt rośnie szybko. Najnowszy nurt (SkillOpt, GEPA) pokazuje, że tę samą pętlę można skierować na same instrukcje agenta i ulepszać je automatycznie, też tylko za zewnętrzną bramą. Jeśli uczysz tego innych albo budujesz z tym produkt, to jest chyba najważniejsza rzecz do zapamiętania: dobra pętla to nie sprytniejszy model, tylko lepsza brama.

Źródła

Recenzowane / arXiv

Wei J. i in. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 (NeurIPS 2022).

Yao S. i in. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 (ICLR 2023).

Yao S. i in. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601 (NeurIPS 2023).

Shinn N. i in. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 (NeurIPS 2023).

Madaan A. i in. (2023). Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651 (NeurIPS 2023).

Gou Z. i in. (2024). CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. arXiv:2305.11738 (ICLR 2024).

Dhuliawala S. i in. (2023). Chain-of-Verification Reduces Hallucination in Large Language Models. arXiv:2309.11495 (ACL Findings 2024).

Huang J. i in. (2023). Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798 (ICLR 2024).

Stechly K., Valmeekam K., Kambhampati S. (2024). On the Self-Verification Limitations of LLMs on Reasoning and Planning Tasks. arXiv:2402.08115.

Agrawal L. i in. (2025). GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv:2507.19457 (ICLR 2026, oral).

Microsoft Research (2026). SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv:2605.23904 (open-source: github.com/microsoft/SkillOpt).

Wytyczne laboratoriów

Anthropic (2024). Building Effective Agents.

Anthropic (2025). Effective Context Engineering for AI Agents. Effective Harnesses for Long-Running Agents. How We Built Our Multi-Agent Research System.

OpenAI (2025). A Practical Guide to Building Agents.

Praktyka / guru

Ng A. (2024). Four Agentic Design Patterns (The Batch, DeepLearning.AI).

Huntley G. (2025). The Ralph Wiggum Loop – walidacja zewnętrzna i świeży kontekst.

Cherny B. / Anthropic (2025). Claude Code: Best Practices for Agentic Coding (research → plan → execute → review → ship).

Strus B. (2026). Agentic loop – działająca pętla, weryfikatory, pięć problemów i benchmark z sekcji 8. Kod otwarty: github.com/blablast/agentic-loop.